

```

// ScoreCI - Proxy API-FOOTBALL
// Garde la cle API cote serveur (jamais exposee au navigateur) et met en cache
// les reponses pour ne pas epuiser le quota (plans gratuits : ~100 requetes/jour
//
// Demarrage : copiez .env.example en .env, renseignez API_FOOTBALL_KEY, puis :
//   npm install
//   npm start

require("dotenv").config();
const express = require("express");
const cors = require("cors");
const fetch = require("node-fetch");
const admin = require("firebase-admin");
const RSSParser = require("rss-parser");
const rssParser = new RSSParser();

const API_KEY = process.env.API_FOOTBALL_KEY;
if (!API_KEY) {
  console.error("[CONFIG] API_FOOTBALL_KEY manquante dans Environment (Render).")
} else if (!/^[a-f0-9]{32}$/i.test(API_KEY)) {
  console.error(`[CONFIG] API_FOOTBALL_KEY suspecte : ${API_KEY.length} caractere
} else {
  console.log(`[CONFIG] API_FOOTBALL_KEY chargee (...${API_KEY.slice(-4)}).`);
}

const PORT = process.env.PORT || 3001;
const ALLOWED_ORIGINS = (process.env.ALLOWED_ORIGINS || "").split(",").map(s => s
const API_BASE = "https://v3.football.api-sports.io";

if (!API_KEY) {
  console.error("ERREUR: API_FOOTBALL_KEY manquante. Copiez .env.example en .env
  process.exit(1);
}

// ---- Firebase Admin (notifications push) ----
// Optionnel : sans cle de service, l'API-FOOTBALL continue de fonctionner
// normalement, seules les routes /api/subscribe et l'envoi de notifs sont
// desactivees.
let messaging = null;
let db = null;
if (process.env.FIREBASE_SERVICE_ACCOUNT_B64) {
  try {
    const json = Buffer.from(process.env.FIREBASE_SERVICE_ACCOUNT_B64, "base64").
    const serviceAccount = JSON.parse(json);
    admin.initializeApp({ credential: admin.credential.cert(serviceAccount) });

```

```

    messaging = admin.messaging();
    db = admin.firestore();
    console.log("Firebase Admin initialise : notifications push activees, Firesto
} catch (err) {
    console.error("Impossible d'initialiser Firebase Admin :", err.message);
}
} else {
    console.log("FIREBASE_SERVICE_ACCOUNT_B64 absente : notifications push desactiv
}

// Jetons des appareils abonnées. Garde en memoire pour la vitesse, mais
// sauvegarde dans Firestore : ne disparaît plus quand le serveur redemarre
// (ce qui arrivait silencieusement avant - les utilisateurs perdaient leurs
// notifications sans le savoir).
const subscribedTokens = new Set();
async function loadTokensFromFirestore() {
    if (!db) return;
    try {
        const snap = await db.collection("push_tokens").get();
        snap.forEach(doc => subscribedTokens.add(doc.id));
        console.log(`${subscribedTokens.size} jeton(s) de notification recharges depu
    } catch (err) {
        console.error("Chargement des jetons Firestore echoue :", err.message);
    }
}
loadTokensFromFirestore();

const app = express();
app.use(cors({
    origin: ALLOWED_ORIGINS.length ? ALLOWED_ORIGINS : true
}));
app.use(express.json({ limit: "16kb" }));

// -----
// Logos « image not available » d'API-Sports -> champ vide.
// L'application affiche alors un monogramme du club a la place de l'appareil pho
// Verification par requetes HEAD sur le CDN media (ne consomme pas le quota API)
// resultats gardes en memoire. Ne bloque jamais une reponse.
// -----
const LOGO_HOST = "https://media.api-sports.io/";
const logoStatus = new Map();           // url -> true (remplacement) | false (vra
const logoQueue = [];
let logoWorkers = 0;
let placeholderSig = null;               // { len, hash } appris a partir des donne
const sizeSeen = new Map();              // taille -> [urls] pendant l'apprentissag
let logoSaveTimer = null;

```

```

function sha1(buf) { return require("crypto").createHash("sha1").update(buf).digest("hex"); }

// Memoire Firestore : survit aux mises en veille de Render.
async function loadLogoMemory() {
  if (!db) { console.log("[LOGOS] Firestore indisponible : apprentissage en memoire");
  try {
    const doc = await db.collection("system").doc("logoPlaceholders").get();
    if (!doc.exists) { console.log("[LOGOS] Aucune memoire : apprentissage en cours");
    const v = doc.data() || {};
    if (v.len && v.hash) placeholderSig = { len: v.len, hash: v.hash };
    (v.urls || []).forEach(u => logoStatus.set(u, true));
    console.log(`[LOGOS] Memoire chargee : ${v.urls || []}.length} logo(s) de rechargement`);
  } catch (e) {
    console.log("[LOGOS] Memoire illisible : " + e.message);
  }
}

function saveLogoMemory() {
  if (!db || !placeholderSig) return;
  clearTimeout(logoSaveTimer);
  logoSaveTimer = setTimeout(() => {
    const urls = [...logoStatus].filter(([, v]) => v === true).map(([u]) => u).slice(0, 3);
    db.collection("system").doc("logoPlaceholders").set({
      len: placeholderSig.len, hash: placeholderSig.hash, urls, updatedAt: Date.now()
    }, { merge: true })
      .catch(e => console.log("[LOGOS] Sauvegarde impossible : " + e.message));
  }, 10000);
}

async function fetchBuffer(url) { const r = await fetch(url, { timeout: 8000 });
  if (!r.ok) return null;
  const buf = await r.arrayBuffer();
  return buf;
}

// L'image « image not available » est identique pour tous les clubs sans logo :
// des que 3 logos differents ont exactement le meme contenu, c'est elle.
async function learnFrom(len, urls) {
  try {
    const hashes = await Promise.all(urls.slice(0, 3).map(async u => sha1(await fetchBuffer(u))));
    if (hashes[0] !== hashes[1] || hashes[1] !== hashes[2]) return false;
    placeholderSig = { len, hash: hashes[0] };
    for (const [l, list] of sizeSeen) list.forEach(u => logoStatus.set(u, l === 1));
    sizeSeen.clear();
    console.log(`[LOGOS] Image de remplacement apprise (${len} octets) : filtre a`);
    saveLogoMemory();
    return true;
  } catch (_) { return false; }
}

async function checkLogo(url) {
  try {
    const buf = await fetchBuffer(url);
    if (!buf) return false;
    const hash = sha1(buf);
    if (placeholderSig.hash === hash) return true;
    return false;
  } catch { return false; }
}

```

```

const head = await fetch(url, { method: "HEAD", timeout: 6000 });
if (!head.ok) { logoStatus.set(url, true); saveLogoMemory(); return; } // 4
let len = Number(head.headers.get("content-length"));
let buf = null;
if (!len) { buf = await fetchBuffer(url); len = buf.length; }
if (placeholderSig) {
  if (len !== placeholderSig.len) { logoStatus.set(url, false); return; }
  buf = buf || await fetchBuffer(url);
  const isPlaceholder = sha1(buf) === placeholderSig.hash;
  logoStatus.set(url, isPlaceholder);
  if (isPlaceholder) saveLogoMemory();
  return;
}
const list = sizeSeen.get(len) || [];
list.push(url); sizeSeen.set(len, list);
logoStatus.set(url, "learning");
if (list.length >= 3 && list.length <= 9 && list.length % 3 === 0) await lear
if (sizeSeen.size > 5000) sizeSeen.clear();
} catch (_) {
  logoStatus.delete(url); // r
}
}

function pumpLogoQueue() {
  while (logoWorkers < 6 && logoQueue.length) {
    const url = logoQueue.shift();
    logoWorkers++;
    checkLogo(url).finally(() => { logoWorkers--; pumpLogoQueue(); });
  }
}

function scrubLogos(node, depth) {
  if (!node || typeof node !== "object" || depth > 8) return;
  if (Array.isArray(node)) { for (const item of node) scrubLogos(item, depth + 1) }
  for (const key of Object.keys(node)) {
    const value = node[key];
    if (typeof value === "string" && /logo$/i.test(key) && value.startsWith(LOGO_)) {
      const status = logoStatus.get(value);
      if (status === true) node[key] = "";
      else if (status === undefined && logoQueue.length < 2000) {
        if (logoStatus.size > 20000) logoStatus.clear();
        logoStatus.set(value, "pending"); logoQueue.push(value);
      }
    } else if (value && typeof value === "object") {
      scrubLogos(value, depth + 1);
    }
  }
}

```

```

}

app.use((req, res, next) => {
  const send = res.json.bind(res);
  res.json = (body) => {
    try { scrubLogos(body, 0); pumpLogoQueue(); } catch (_) {}
    return send(body);
  };
  next();
});

app.disable("x-powered-by");
app.use(express.static(require("path").join(__dirname, "public")));
const adminNotifySecret = process.env.NOTIFY_ADMIN_SECRET;
function requireNotifyAdmin(req, res, next) {
  const supplied = Buffer.from(req.get("authorization") || "");
  const expected = Buffer.from("Bearer " + (adminNotifySecret || ""));
  if (!adminNotifySecret || supplied.length !== expected.length ||
    !require("crypto").timingSafeEqual(supplied, expected)) {
    return res.status(401).json({ error: "Authentification administrateur requise" });
  }
  next();
}

// ---- Competitions exposees a ScoreCI ----
// Chaque entree decrit comment RETROUVER l'id numerique API-FOOTBALL de la
// competition (on ne code pas les id en dur : on les resout une fois via
// /leagues puis on les met en cache, pour eviter toute erreur de mapping).
const COMPETITIONS = {
  liguelci: { search: "Ligue 1", country: "Ivory-Coast" },
  can: { search: "Africa Cup of Nations", country: null },
  cafcl: { search: "CAF Champions League", country: null },
  qualifs: { search: "World Cup - Qualification Africa", country: null },
  ucl: { search: "UEFA Champions League", country: null },
  uel: { search: "UEFA Europa League", country: null },
  llfr: { search: "Ligue 1", country: "France" },
  pl: { search: "Premier League", country: "England" },
  liga: { search: "La Liga", country: "Spain" },
  seriea: { search: "Serie A", country: "Italy" },
  bundesliga: { search: "Bundesliga", country: "Germany" },
  eredivisie: { search: "Eredivisie", country: "Netherlands" },
  primeira: { search: "Primeira Liga", country: "Portugal" },
  superlig: { search: "Super Lig", country: "Turkey" },
  saudipl: { search: "Pro League", country: "Saudi-Arabia" },
  mls: { search: "Major League Soccer", country: "USA" }
};

// Ordre d'affichage : grands championnats d'abord, le reste ensuite

```

```

// (utilise pour trier les groupes de matchs en direct).
const LEAGUE_PRIORITY = [
  "World Cup - Qualification Africa", "Africa Cup of Nations", "CAF Champions Lea
  "UEFA Champions League", "UEFA Europa League",
  "Premier League", "La Liga", "Serie A", "Bundesliga", "Ligue 1",
  "Primeira Liga", "Eredivisie", "Super Lig"
];

function leaguePriority(name) {
  const idx = LEAGUE_PRIORITY.indexOf(name);
  return idx === -1 ? 999 : idx;
}

// ---- Cache memoire simple (TTL) ----
const cache = new Map();
function cacheGet(key) {
  const hit = cache.get(key);
  if (!hit) return null;
  if (Date.now() > hit.expires) { cache.delete(key); return null; }
  return hit.value;
}
function cacheSet(key, value, ttlMs) {
  if (cache.size >= 2000) cache.delete(cache.keys().next().value);
  cache.set(key, { value, expires: Date.now() + ttlMs });
}

// ---- Suivi du quota API-FOOTBALL (compteur local + coupe-circuit) ----
// Le tableau de bord API-FOOTBALL a montre des pics proches de 150 000/jour
// (le plafond du plan Mega). Ce compteur local nous permet de reagir AVANT
// d'epuiser le quota, plutot que de le decouvrir apres coup.
const DAILY_LIMIT = Math.max(1, Number(process.env.API_DAILY_LIMIT) || 150000);
const SAFETY_THRESHOLD = 0.85; // 85% : on coupe les taches de fond, on garde l'e
let requestCount = 0;
let countResetAt = nextUtcMidnight();
const requestsByEndpoint = {};

function nextUtcMidnight() {
  const d = new Date();
  d.setUTCHours(24, 0, 0, 0);
  return d;
}

function trackRequest(path) {
  if (Date.now() > countResetAt.getTime()) {
    requestCount = 0;
    for (const k in requestsByEndpoint) delete requestsByEndpoint[k];
    countResetAt = nextUtcMidnight();
  }
  requestCount++;
}

```

```

const endpoint = path.split("?")[0];
requestsByEndpoint[endpoint] = (requestsByEndpoint[endpoint] || 0) + 1;
if (requestCount === Math.floor(DAILY_LIMIT * SAFETY_THRESHOLD)) {
  console.warn(`ATTENTION : ${requestCount} requetes API-FOOTBALL aujourd'hui (
}
}
function quotaSafetyOk() {
  if (Date.now() >= countResetAt.getTime()) {
    requestCount = 0;
    for (const key in requestsByEndpoint) delete requestsByEndpoint[key];
    countResetAt = nextUtcMidnight();
  }
  return requestCount < DAILY_LIMIT * SAFETY_THRESHOLD;
}

// Fusionne les appels simultanes identiques ; ne conserve pas les erreurs.
const pendingApiRequests = new Map();
async function apiGet(path) {
  if (pendingApiRequests.has(path)) return pendingApiRequests.get(path);
  const promise = (async () => {
    quotaSafetyOk();
    if (requestCount >= DAILY_LIMIT) throw new Error("Quota API journalier attein
trackRequest(path);
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), 12000);
    try {
      const res = await fetch(`${API_BASE}${path}`, {
        headers: { "x-apisports-key": API_KEY }, signal: controller.signal
      });
      if (!res.ok) {
        // Journalise la raison exacte renvoyee par API-FOOTBALL (jamais la cle).
        let body = "";
        try { body = (await res.text()).slice(0, 300); } catch (_) {}
        console.error(`[API-FOOTBALL] HTTP ${res.status} sur ${path.split("?")[0]
        throw new Error(`API-FOOTBALL HTTP ${res.status}`);
      }
      const json = await res.json();
      if (json.errors && Object.keys(json.errors).length) {
        const reason = JSON.stringify(json.errors).slice(0, 300);
        console.error(`[API-FOOTBALL] Erreur fournisseur sur ${path.split("?")[0]
        throw new Error("Erreur fournisseur API-FOOTBALL");
      }
      return json.response;
    } finally { clearTimeout(timer); }
  })();
  pendingApiRequests.set(path, promise);
  try { return await promise; }

```

```

    finally { pendingApiRequests.delete(path); }
}

// Resout et met en cache l'id de ligue + la saison en cours pour une competition
async function resolveLeague(compId) {
  const cacheKey = `league:${compId}`;
  const cached = cacheGet(cacheKey);
  if (cached) return cached;

  const cfg = COMPETITIONS[compId];
  if (!cfg) throw new Error(`Competition inconnue: ${compId}`);

  let qs = `search=${encodeURIComponent(cfg.search)}`;
  const results = await apiGet(`/leagues?${qs}`);
  if (!results || !results.length) {
    throw new Error(`Aucune ligue trouvee pour "${cfg.search}"`);
  }
  // Si un pays est precise, on exige une correspondance exacte (sinon on
  // risquerait de resoudre "Ligue 1" vers la France au lieu de la CI, par
  // exemple) ; sinon on prend le 1er resultat.
  let match;
  if (cfg.country) {
    match = results.find(r => r.country && r.country.name.replace(/[ -]/g, "").to
    if (!match) {
      throw new Error(`Ligue "${cfg.search}" introuvable pour le pays "${cfg.coun
    }
  } else {
    match = results[0];
  }

  const activeSeason = (match.seasons || []).find(s => s.current);
  const fallbackSeason = (match.seasons || []).slice(-1)[0];
  const selectedSeason = activeSeason || fallbackSeason;
  const info = {
    leagueId: match.league.id,
    season: selectedSeason ? selectedSeason.year : new Date().getFullYear(),
    name: match.league.name,
    current: Boolean(activeSeason)
  };
  cacheSet(cacheKey, info, 24 * 60 * 60 * 1000); // 24h : les ids ne changent pas
  return info;
}

// GET /api/standings/:comp -> classement normalise pour ScoreCI
app.get("/api/standings/:comp", async (req, res) => {
  const compId = req.params.comp;
  if (!COMPETITIONS[compId]) return res.status(404).json({ error: "Competition in

```

```

const cacheKey = `standings:${compId}`;
const cached = cacheGet(cacheKey);
if (cached) return res.json(cached);

try {
  const { leagueId, season, name, current } = await resolveLeague(compId);
  if (!current) {
    return res.status(409).json({ error: "Compétition non active actuellement",
  }
  const response = await apiGet(`/standings?league=${leagueId}&season=${season}`);

  const groups = response?.[0]?.league?.standings || [];
  // Une compétition peut avoir plusieurs groupes/poules (CAN, qualifs...).
  // On renvoie TOUS les groupes (avec leur label), le front les affiche
  // séparément avec un en-tête par groupe plutôt que de les fusionner
  // (ce qui mélangeait les classements) ou de n'en garder qu'un seul
  // (ce qui cachait les autres poules).
  const ciGroupIdx = groups.findIndex(g => g.some(row => /ivoire|ivory coast/i).
  const orderedGroups = ciGroupIdx > 0
    ? [groups[ciGroupIdx], ...groups.slice(0, ciGroupIdx), ...groups.slice(ciGr
      : groups;

  const groupPayloads = orderedGroups.map(g => ({
    label: g[0]?.group || null,
    teams: g.map(row => ({
      emoji: null,
      logo: row.team.logo,
      name: row.team.name,
      color: null,
      j: row.all.played,
      g: row.all.win,
      n: row.all.draw,
      d: row.all.lose,
      pts: row.points,
      zone: row.description || null // ex: "Promotion - Champions League", "Rel
    })))
  }));

  const payload = { comp: compId, label: name, season, groups: groupPayloads, u
  cacheSet(cacheKey, payload, 3 * 60 * 1000); // 3 min (plan Mega : tres large
  res.json(payload);
} catch (err) {
  console.error(err.message);
  res.status(502).json({ error: "Impossible de récupérer le classement", detail
}
});

```

```

// GET /api/day-fixtures?date=YYYY-MM-DD
// Flux principal ScoreCI : tous les matchs reels de la journee, sans se limiter
// a une liste de competitions. C'est cette route qui alimente Hier/Aujourd'hui/D
app.get("/api/day-fixtures", async (req, res) => {
  const day = req.query.date || "";
  if (typeof day !== "string" || !/^\\d{4}-\\d{2}-\\d{2}$/.test(day) ||
    !Number.isFinite(Date.parse(day)) || new Date(day).toISOString().slice(0, 1
    return res.status(400).json({ error: "Date invalide" });
  }
  const cacheKey = `day-feed:${day}`;
  const cached = cacheGet(cacheKey);
  if (cached) return res.json(cached);

  try {
    const response = await apiGet(`/fixtures?date=${day}&timezone=Africa%2FAbidja
    if (!Array.isArray(response)) throw new Error("Reponse fournisseur invalide")
    const matches = response.map(f => ({
      id: f.fixture.id,
      date: f.fixture.date,
      status: f.fixture.status.short,
      elapsed: f.fixture.status.elapsed,
      home: f.teams.home.name,
      homeId: f.teams.home.id,
      away: f.teams.away.name,
      awayId: f.teams.away.id,
      homeLogo: f.teams.home.logo,
      awayLogo: f.teams.away.logo,
      homeScore: f.goals.home,
      awayScore: f.goals.away,
      comp: f.league.name,
      compId: f.league.id,
      compLogo: f.league.logo,
      country: f.league.country || "",
      flag: f.league.flag || null,
      season: f.league.season,
      round: f.league.round
    })));

    const groupsMap = new Map();
    matches.forEach(m => {
      const key = `${m.compId}:${m.comp}`;
      if (!groupsMap.has(key)) groupsMap.set(key, {
        competition: m.comp,
        competitionId: m.compId,
        logo: m.compLogo,
        country: m.country,

```

```

        flag: m.flag,
        matches: []
    });
    groupsMap.get(key).matches.push(m);
});
const groups = Array.from(groupsMap.values()).sort((a, b) => {
    const ap = leaguePriority(a.competition), bp = leaguePriority(b.competition)
    if (ap !== bp) return ap - bp;
    if (a.country === "Ivory-Coast" && b.country !== "Ivory-Coast") return -1;
    if (b.country === "Ivory-Coast" && a.country !== "Ivory-Coast") return 1;
    return a.competition.localeCompare(b.competition);
});
const payload = { date: day, matches, groups, updatedAt: new Date().toISOString() };
cacheSet(cacheKey, payload, 60 * 1000);
res.json(payload);
} catch (err) {
    console.error("day fixtures:", err.message);
    res.status(502).json({ error: "Impossible de recuperer les matchs de la journ"
});

// GET /api/fixtures/:comp?scope=results|upcoming
app.get("/api/fixtures/:comp", async (req, res) => {
    const compId = req.params.comp;
    const scope = req.query.scope === "upcoming" ? "upcoming" : "results";
    if (!COMPETITIONS[compId]) return res.status(404).json({ error: "Competition in

const day = req.query.date || "";
if (day && (typeof day !== "string" || !/^\\d{4}-\\d{2}-\\d{2}$/.test(day) ||
    !Number.isFinite(Date.parse(day)) || new Date(day).toISOString().slice(0, 1
    return res.status(400).json({ error: "Date invalide" });
const cacheKey = `fixtures:${compId}:${scope}:${day}`;
const cached = cacheGet(cacheKey);
if (cached) return res.json(cached);

try {
    const { leagueId, season, name } = await resolveLeague(compId);
    const param = scope === "upcoming" ? "next" : "last";
    const query = day ? `date=${day}&timezone=UTC` : `league=${leagueId}&season=$
    // A daily query spans seasons. Reuse it across competitions, then filter
    // by the resolved league ID instead of combining league with no season.
    const dailyKey = `fixtures-day:${day}`;
    let response = day ? cacheGet(dailyKey) : null;
    if (!response) {
        response = await apiGet(`/fixtures?${query}`);
        if (!Array.isArray(response)) throw new Error("Reponse fournisseur invalide
        if (day) cacheSet(dailyKey, response, 2 * 60 * 1000);

```

```

    }
    if (day) response = response.filter(f => String(f.league.id) === String(leagu

const matches = response.map(f => ({
    id: f.fixture.id,
    date: f.fixture.date,
    status: f.fixture.status.short,
    home: f.teams.home.name,
    homeId: f.teams.home.id,
    away: f.teams.away.name,
    awayId: f.teams.away.id,
    homeLogo: f.teams.home.logo,
    awayLogo: f.teams.away.logo,
    homeScore: f.goals.home,
    awayScore: f.goals.away,
    comp: name,
    compId: f.league.id,
    season: f.league.season,
    round: f.league.round,
    elapsed: f.fixture.status.elapsed
})));

const payload = { comp: compId, scope, name, season, date: day || null, match
cacheSet(cacheKey, payload, 2 * 60 * 1000); // 2 min
res.json(payload);
} catch (err) {
    console.error(err.message);
    res.status(502).json({ error: "Impossible de recuperer les matchs", detail: e
    }
});

app.get("/api/health", (req, res) => res.json({ ok: true }));

// GET /api/quota-status -> suivi du quota API-FOOTBALL, sans passer par leur sit
app.get("/api/quota-status", (req, res) => {
    if (Date.now() > countResetAt.getTime()) {
        requestCount = 0;
        for (const k in requestsByEndpoint) delete requestsByEndpoint[k];
        countResetAt = nextUtcMidnight();
    }
    res.json({
        used: requestCount,
        limit: DAILY_LIMIT,
        percent: Math.round((requestCount / DAILY_LIMIT) * 1000) / 10,
        safetyOk: quotaSafetyOk(),
        resetsAt: countResetAt.toISOString(),
        byEndpoint: requestsByEndpoint
    });
});

```

```

    });
});

// GET /api/live -> tous les matchs en direct, regroupes par championnat
// Fonction partagee : recupere les matchs en direct, regroupes par championnat.
// Utilisee a la fois par /api/live (vue utilisateur) ET par la detection de
// buts en arriere-plan, pour ne jamais faire deux appels API distincts pour
// la meme donnee (le quota gratuit est de 100 requetes/jour seulement).
async function getLiveFixtures() {
    const cacheKey = "live:all";
    const cached = cacheGet(cacheKey);
    if (cached) return cached;

    const response = await apiGet(`/fixtures?live=all`);
    const flat = (response || []).map(f => ({
        id: f.fixture.id,
        date: f.fixture.date,
        status: f.fixture.status.short,
        elapsed: f.fixture.status.elapsed,
        home: f.teams.home.name,
        homeId: f.teams.home.id,
        away: f.teams.away.name,
        awayId: f.teams.away.id,
        homeLogo: f.teams.home.logo,
        awayLogo: f.teams.away.logo,
        homeScore: f.goals.home,
        awayScore: f.goals.away,
        comp: f.league.name,
        compId: f.league.id,
        country: f.league.country,
        compLogo: f.league.logo,
        season: f.league.season,
        round: f.league.round
    }));
    const groups = {};
    flat.forEach(m => {
        const key = m.comp + (m.country ? " (" + m.country + ")" : "");
        if (!groups[key]) groups[key] = { competition: m.comp, country: m.country, lo
        groups[key].matches.push(m);
    });
    const payload = {
        matches: flat,
        groups: Object.values(groups).sort((a, b) => leaguePriority(a.competition) -
        updatedAt: new Date().toISOString()
    };
    cacheSet(cacheKey, payload, 15 * 1000); // 15s : quasi temps reel (plan Mega)
    return payload;

```

```

}

app.get("/api/live", async (req, res) => {
  try {
    const payload = await getLiveFixtures();
    res.json(payload);
  } catch (err) {
    console.error(err.message);
    res.status(502).json({ error: "Impossible de recuperer les matchs en direct",
  }
});

// GET /api/news?zone=ci|afrique|monde -> actualites football en temps reel
// via Google News RSS (meme principe que le backend KSB Sport).
const NEWS_QUERIES = {
  ci: [
    'football "Côte d Ivoire" when:7d',
    '"Éléphants" football when:7d',
    '"ASEC Mimosas" when:7d',
    '"Africa Sports" football when:7d',
    '"Ligue 1" "Côte d Ivoire" football when:7d'
  ],
  afrique: [
    'football Afrique CAF when:7d',
    '"CAF Champions League" football when:7d',
    '"Coupe de la Confédération CAF" football when:7d',
    'sélections africaines football when:7d',
    'mercato joueurs africains football when:7d'
  ],
  monde: [
    '"Champions League" football when:7d',
    '"Premier League" football when:7d',
    '"La Liga" football when:7d',
    '"Serie A" football when:7d',
    'Bundesliga football when:7d',
    '"Ligue 1" football France when:7d',
    'FIFA football international when:7d',
    'mercato football Europe when:7d'
  ]
};

app.get("/api/news", async (req, res) => {
  const zone = ["ci", "afrique", "monde"].includes(req.query.zone) ? req.query.zo
  const cacheKey = `news:v3:${zone}`;
  const cached = cacheGet(cacheKey);
  if (cached) return res.json(cached);

  try {

```

```

const queries = NEWS_QUERIES[zone] || NEWS_QUERIES.ci;
const feeds = await Promise.all(queries.map(async query => {
  const q = encodeURIComponent(query);
  const url = `https://news.google.com/rss/search?q=${q}&hl=fr&gl=CI&ceid=CI:
  try { return await rssParser.parseURL(url); }
  catch (e) { console.warn('news feed:', query, e.message); return { items: [
}));

const seen = new Set();
const now = Date.now();
const minDate = now - 7 * 24 * 60 * 60 * 1000;
const items = [];
feeds.forEach(feed => {
  (feed.items || []).forEach(it => {
    const dateValue = it.isoDate || it.pubDate;
    const ts = Date.parse(dateValue || '');
    if (!Number.isFinite(ts) || ts < minDate || ts > now + 5 * 60 * 1000) ret
    const rawTitle = String(it.title || '').trim();
    const key = rawTitle.toLowerCase().replace(/\s+/g, ' ');
    if (!key || seen.has(key)) return;
    seen.add(key);
    items.push({
      title: rawTitle,
      link: it.link,
      date: dateValue,
      source: rawTitle.includes(' - ') ? rawTitle.split(' - ').pop() : (it.cr
      summary: it.contentSnippet || it.content || it.summary || ''
    });
  });
});
items.sort((a, b) => Date.parse(b.date) - Date.parse(a.date));
const payload = { zone, items: items.slice(0, 50), updatedAt: new Date().toIS
cacheSet(cacheKey, payload, 5 * 60 * 1000);
res.json(payload);
} catch (err) {
  console.error("news:", err.message);
  res.status(502).json({ error: "Impossible de recuperer les actualites", detai
}
});

// POST /api/subscribe { token } -> enregistre un appareil pour les notifications
// GET /api/fixture/:id -> detail complet d'un match (evenements, stade, arbitre)
app.get("/api/fixture/:id", async (req, res) => {
  const id = req.params.id;
  const cacheKey = `fixture:${id}`;
  const cached = cacheGet(cacheKey);
  if (cached) return res.json(cached);

```

```

try {
  const [response, playersResponse] = await Promise.all([
    apiGet(`/fixtures?id=${id}`),
    apiGet(`/fixtures/players?fixture=${id}`).catch(() => [])
  ]);
  const f = response && response[0];
  if (!f) return res.status(404).json({ error: "Match introuvable" });

  // Note de performance par joueur (comme les badges colores type Sofascore)
  const ratings = {};
  (playersResponse || []).forEach(teamBlock => {
    (teamBlock.players || []).forEach(p => {
      const stat = p.statistics && p.statistics[0];
      const r = stat && stat.games && stat.games.rating;
      if (r) ratings[p.player.id] = parseFloat(r);
    });
  });

  const events = (f.events || []).map(e => ({
    minute: e.time.elapsed + (e.time.extra ? "+" + e.time.extra : ""),
    type: e.type, // "Goal", "Card", "subst"...
    detail: e.detail, // "Normal Goal", "Yellow Card"...
    team: e.team.name,
    player: e.player.name,
    assist: e.assist ? e.assist.name : null
  }));

  // Compositions (si disponibles pour ce match)
  const lineups = (f.lineups || []).map(l => ({
    team: l.team.name,
    teamLogo: l.team.logo,
    formation: l.formation,
    coach: l.coach ? l.coach.name : null,
    startXI: (l.startXI || []).map(p => ({
      id: p.player.id, name: p.player.name, number: p.player.number,
      pos: p.player.pos, grid: p.player.grid,
      rating: ratings[p.player.id] || null,
      photo: `https://media.api-sports.io/football/players/${p.player.id}.png`
    })),
    substitutes: (l.substitutes || []).map(p => ({
      id: p.player.id, name: p.player.name, number: p.player.number, pos: p.pla
      rating: ratings[p.player.id] || null
    })))
  }));

  // Statistiques (possession, tirs, corners...)

```

```

const statistics = (f.statistics || []).map(s => ({
  team: s.team.name,
  stats: (s.statistics || []).map(st => ({ type: st.type, value: st.value }))
}));

const payload = {
  id: f.fixture.id,
  date: f.fixture.date,
  status: f.fixture.status.short,
  elapsed: f.fixture.status.elapsed,
  venue: f.fixture.venue ? f.fixture.venue.name : null,
  referee: f.fixture.referee || null,
  home: f.teams.home.name,
  away: f.teams.away.name,
  homeId: f.teams.home.id,
  awayId: f.teams.away.id,
  homeLogo: f.teams.home.logo,
  awayLogo: f.teams.away.logo,
  homeScore: f.goals.home,
  awayScore: f.goals.away,
  comp: f.league.name,
  compId: f.league.id,
  country: f.league.country,
  season: f.league.season,
  round: f.league.round,
  events,
  lineups,
  statistics,
  updatedAt: new Date().toISOString()
};

cacheSet(cacheKey, payload, 15 * 1000); // 15s : peut evoluer si le match est
res.json(payload);
} catch (err) {
  console.error("fixture detail:", err.message);
  res.status(502).json({ error: "Impossible de recuperer le detail du match", d
  }
});

// GET /api/h2h?team1=ID&team2=ID -> historique des confrontations
app.get("/api/h2h", async (req, res) => {
  const { team1, team2 } = req.query;
  if (!team1 || !team2) return res.status(400).json({ error: "team1 et team2 requ
  const cacheKey = `h2h:${team1}-${team2}`;
  const cached = cacheGet(cacheKey);
  if (cached) return res.json(cached);

  try {

```

```

const response = await apiGet(`/fixtures/headtohead?h2h=${team1}-${team2}&las
const matches = (response || []).map(f => ({
  date: f.fixture.date,
  home: f.teams.home.name,
  away: f.teams.away.name,
  homeScore: f.goals.home,
  awayScore: f.goals.away,
  comp: f.league.name
})));
const payload = { matches, updatedAt: new Date().toISOString() };
cacheSet(cacheKey, payload, 60 * 60 * 1000); // 1h : l'historique bouge peu
res.json(payload);
} catch (err) {
  console.error("h2h:", err.message);
  res.status(502).json({ error: "Impossible de recuperer le face-a-face", detai
}
});

// GET /api/team/:id -> fiche club (infos, stade, effectif)
app.get("/api/team/:id", async (req, res) => {
  const id = req.params.id;
  const cacheKey = `team:${id}`;
  const cached = cacheGet(cacheKey);
  if (cached) return res.json(cached);

  try {
    const [teamRes, squadRes] = await Promise.all([
      apiGet(`/teams?id=${id}`),
      apiGet(`/players/squads?team=${id}`)
    ]);
    const t = teamRes && teamRes[0];
    if (!t) return res.status(404).json({ error: "Club introuvable" });
    const squad = (squadRes && squadRes[0] && squadRes[0].players || []).map(p =>
      id: p.id, name: p.name, age: p.age, number: p.number, position: p.position,
    ));
    const payload = {
      id: t.team.id,
      name: t.team.name,
      logo: t.team.logo,
      country: t.team.country,
      founded: t.team.founded,
      venue: t.venue ? { name: t.venue.name, city: t.venue.city, capacity: t.venu
      squad,
      updatedAt: new Date().toISOString()
    };
    cacheSet(cacheKey, payload, 24 * 60 * 60 * 1000); // 24h : infos club stables
    res.json(payload);
  }
});

```

```

    } catch (err) {
      console.error("team detail:", err.message);
      res.status(502).json({ error: "Impossible de recuperer le club", detail: err.
    }
  });

// GET /api/player/:id -> fiche joueur (profil + stats saison)
app.get("/api/player/:id", async (req, res) => {
  const id = req.params.id;
  const cacheKey = `player:${id}`;
  const cached = cacheGet(cacheKey);
  if (cached) return res.json(cached);

  try {
    const season = new Date().getFullYear();
    const response = await apiGet(`/players?id=${id}&season=${season}`);
    const p = response && response[0];
    if (!p) return res.status(404).json({ error: "Joueur introuvable" });
    const st = (p.statistics && p.statistics[0]) || {};
    const payload = {
      id: p.player.id,
      name: p.player.name,
      photo: p.player.photo,
      age: p.player.age,
      nationality: p.player.nationality,
      height: p.player.height,
      weight: p.player.weight,
      team: st.team ? st.team.name : null,
      teamLogo: st.team ? st.team.logo : null,
      position: st.games ? st.games.position : null,
      appearances: st.games ? st.games.appearances : null,
      goals: st.goals ? st.goals.total : null,
      assists: st.goals ? st.goals.assists : null,
      yellowCards: st.cards ? st.cards.yellow : null,
      redCards: st.cards ? st.cards.red : null,
      updatedAt: new Date().toISOString()
    };
    cacheSet(cacheKey, payload, 6 * 60 * 60 * 1000); // 6h
    res.json(payload);
  } catch (err) {
    console.error("player detail:", err.message);
    res.status(502).json({ error: "Impossible de recuperer le joueur", detail: er
  }
});

// GET /api/search?q=... -> recherche reelle clubs + joueurs
app.get("/api/search", async (req, res) => {

```

```

const q = (req.query.q || "").trim();
if (q.length < 3) return res.json({ teams: [], players: [] });
const cacheKey = `search:${q.toLowerCase()}`;
const cached = cacheGet(cacheKey);
if (cached) return res.json(cached);

try {
  const [teamsRes, playersRes] = await Promise.all([
    apiGet(`/teams?search=${encodeURIComponent(q)}`).catch(() => []),
    apiGet(`/players/profiles?search=${encodeURIComponent(q)}`).catch(() => [])
  ]);
  const teams = (teamsRes || []).slice(0, 8).map(t => ({
    id: t.team.id, name: t.team.name, logo: t.team.logo, country: t.team.countr
  }));
  const players = (playersRes || []).slice(0, 8).map(p => ({
    id: p.player.id, name: p.player.name, photo: p.player.photo, nationality: p
  }));
  const payload = { teams, players };
  cacheSet(cacheKey, payload, 30 * 60 * 1000); // 30 min
  res.json(payload);
} catch (err) {
  console.error("search:", err.message);
  res.status(502).json({ error: "Recherche indisponible", detail: err.message }
}
});

// GET /api/leagues/search?q=... -> parcourir N'IMPORTE QUEL championnat au monde
// (pas seulement les 15 pre-configures), regroupes par pays comme sur Sofascore.
app.get("/api/leagues/search", async (req, res) => {
  const q = (req.query.q || "").trim();
  const cacheKey = `leaguesearch:${q.toLowerCase()}`;
  const cached = cacheGet(cacheKey);
  if (cached) return res.json(cached);

  try {
    const path = q.length >= 2 ? `/leagues?search=${encodeURIComponent(q)}` : `/l
    const response = await apiGet(path);
    const grouped = {};
    (response || []).slice(0, 200).forEach(item => {
      const currentSeason = (item.seasons || []).find(s => s.current);
      if (!currentSeason) return; // ignore les saisons terminees/futures
      const country = item.country ? item.country.name : "International";
      if (!grouped[country]) grouped[country] = { country, flag: item.country ? i
      grouped[country].leagues.push({
        id: item.league.id,
        name: item.league.name,
        logo: item.league.logo,

```

```

        type: item.league.type,
        season: currentSeason.year
    });
});
const countries = Object.values(grouped).sort((a, b) => a.country.localeCompa
const payload = { countries };
cacheSet(cacheKey, payload, 6 * 60 * 60 * 1000); // 6h
res.json(payload);
} catch (err) {
    console.error("leagues search:", err.message);
    res.status(502).json({ error: "Recherche de championnats indisponible", detai
}
});

// GET /api/standings/by-id/:leagueId?season=YYYY -> classement de N'IMPORTE
// QUEL championnat par son id (pour la navigation libre ci-dessus).
app.get("/api/standings/by-id/:leagueId", async (req, res) => {
    const leagueId = req.params.leagueId;
    const season = req.query.season || new Date().getFullYear();
    const cacheKey = `standings-id:${leagueId}:${season}`;
    const cached = cacheGet(cacheKey);
    if (cached) return res.json(cached);

    try {
        const response = await apiGet(`/standings?league=${leagueId}&season=${season}`
        // Le endpoint standings peut renvoyer un tableau vide pour les coupes,
        // qualifications ou competitions non couvertes. Dans ce cas, recuperer
        // quand meme le vrai nom de la competition au lieu d'afficher
        // le generique "Championnat".
        let leagueName = response?.[0]?.league?.name || null;
        if (!leagueName) {
            try {
                const leagueInfo = await apiGet(`/leagues?id=${encodeURIComponent(leagueI
                leagueName = leagueInfo?.[0]?.league?.name || null;
            } catch (_) {}
        }
        leagueName = leagueName || "Compétition";
        const groups = response?.[0]?.league?.standings || [];
        const groupPayloads = groups.map(g => ({
            label: g[0]?.group || null,
            teams: g.map(row => ({
                emoji: null, logo: row.team.logo, name: row.team.name, color: null,
                j: row.all.played, g: row.all.win, n: row.all.draw, d: row.all.lose,
                pts: row.points, zone: row.description || null
            })))
        }));
        const payload = { label: leagueName, season, groups: groupPayloads, updatedAt

```

```

        cacheSet(cacheKey, payload, 10 * 60 * 1000); // 10 min
        res.json(payload);
    } catch (err) {
        console.error("standings by id:", err.message);
        res.status(502).json({ error: "Impossible de recuperer ce classement", detail
    }
});

app.post("/api/subscribe", async (req, res) => {
    const { token } = req.body || {};
    if (!messaging || !db) return res.status(503).json({ error: "Notifications non
    if (typeof token !== "string" || token.length < 20 || token.length > 4096 || /[
        return res.status(400).json({ error: "Jeton invalide" });
    subscribedTokens.add(token);
    if (db) {
        try {
            await db.collection("push_tokens").doc(token).set({ subscribedAt: new Date(
        } catch (err) {
            subscribedTokens.delete(token);
            return res.status(503).json({ error: "Enregistrement impossible, veuillez r
        }
    }
    res.json({ ok: true, total: subscribedTokens.size });
});

// POST /api/notify { title, body } -> envoie une notif a tous les appareils abon
// (route manuelle, utile pour tester ; la detection automatique de buts
// utilise la meme fonction en interne, voir plus bas)
async function sendNotification(title, body) {
    if (!messaging || !subscribedTokens.size) return { sent: 0 };
    const tokens = Array.from(subscribedTokens);
    let sent = 0;
    for (let offset = 0; offset < tokens.length; offset += 500) {
        const batch = tokens.slice(offset, offset + 500);
        const result = await messaging.sendEachForMulticast({ tokens: batch,
            notification: { title, body }, webpush: { fcmOptions: { link: "/" } } });
        sent += result.successCount;
        result.responses.forEach((r, i) => {
            if (!r.success && r.error && ["messaging/registration-token-not-registered"
                "messaging/invalid-registration-token"].includes(r.error.code)) {
                subscribedTokens.delete(batch[i]);
                if (db) db.collection("push_tokens").doc(batch[i]).delete().catch(() => {
            }
        });
    }
    return { sent };
}

```

```

app.post("/api/notify", requireNotifyAdmin, async (req, res) => {
  if (!messaging) return res.status(503).json({ error: "Notifications non configu
  const { title, body } = req.body || {};
  if (!title || !body) return res.status(400).json({ error: "title et body requis
  try {
    const result = await sendNotification(title, body);
    res.json(result);
  } catch (err) {
    console.error(err.message);
    res.status(502).json({ error: "Envoi echoue", detail: err.message });
  }
});

// ---- Detection automatique de buts sur les matchs en direct ----
// Verification toutes les 20 secondes lorsque des appareils sont abonnées.
// Le cache live est partage ; ajuster le quota selon le contrat fournisseur.
const lastScores = new Map(); // fixtureId -> "home-away"
async function checkLiveGoals() {
  if (!messaging || !subscribedTokens.size) return;
  if (!quotaSafetyOk()) return; // coupe-circuit : on protege le quota du jour
  try {
    const data = await getLiveFixtures();
    (data.matches || []).forEach(f => {
      const score = `${f.homeScore ?? 0}-${f.awayScore ?? 0}`;
      const prev = lastScores.get(f.id);
      if (prev && prev !== score) {
        sendNotification(
          (score.split("-").reduce((a, b) => a + Number(b), 0) > prev.split("-").
            `${f.home} ${score} ${f.away} (${f.comp})`
        ).catch(e => console.error("Notif echouee:", e.message));
      }
      lastScores.set(f.id, score);
    });
  } catch (err) {
    console.error("checkLiveGoals:", err.message);
  }
}
setInterval(checkLiveGoals, 20 * 1000); // plan Mega : detection de buts quasi in

app.listen(PORT, () => {
  console.log(`ScoreCI API proxy en ecoute sur le port ${PORT}`);
  loadLogoMemory();
});

```